

Protein Circuit Topology Plugin - Complete API Documentation

Table of Contents

1. [Calculating Functions](#) (6 functions)
2. [Plotting Functions](#) (6 functions)
3. [Importing Functions](#) (1 functions)
4. [Exporting Functions](#) (3 functions)
5. [Analysis Functions](#) (6 functions)
6. [Utility Functions](#) (41 functions)
7. [GUI Functions](#) (44 functions)
8. [Initialization Functions](#) (11 functions)

Calculating Functions

```
energy_cmap(index, numbering, res_names, protid, potential_sign='-')
```

Module: `functions/calculating/energy_cmap.py`

Applies an energy filter to an existing residue contact map based on a potential matrix.

Args:

- `index` (`numpy.ndarray`): Array of contact indices.
- `numbering` (`list` or `numpy.ndarray`): List of residue numbers/identifiers.
- `res_names` (`list`): List of residue names corresponding to the numbering.
- `protid` (`str`): Protein identifier.
- `potential_sign` (`str`, optional): The sign of the potential to filter by (`'+'` or `'-'`). Defaults to `'-'`.

Returns: `tuple` — Filtered contact indices (`numpy.ndarray`) and the updated protein identifier (`str`).

```
get_cmap(chain, level='chain', cutoff_distance=4.5, cutoff_numcontacts=5, exclude_neighbour=3)
```

Module: `functions/calculating/get_cmap.py`

Creates a residue-residue contact map (as a list of contacts) for a single chain or a whole model.

Args:

- `chain` (`Bio.PDB.Chain.Chain` or `Bio.PDB.Model.Model`): The chain or model object to analyze.
- `level` (`str`, optional): `'chain'` for single chain analysis, `'model'` for whole model analysis. Defaults to `'chain'`.
- `cutoff_distance` (`float`, optional): Maximum distance (in Å) between atoms to consider a contact. Defaults to `4.5`.
- `cutoff_numcontacts` (`int`, optional): Minimum number of atomic contacts required to define a residue-residue contact. Defaults to `5`.
- `exclude_neighbour` (`int`, optional): Minimum sequence separation (in residues) to consider a contact. Defaults to `3`.

Returns: `tuple` — (`index`, `numbering`, `protid`, `res_names`) where `index` is a `numpy.ndarray` of contact indices, `numbering` is a `numpy.ndarray` of residue numbers, `protid` is a `str`, and `res_names` is a `list` of residue names.

```
get_matrix(index, protid)
```

Module: `functions/calculating/get_matrix.py`

Creates a topological relationship matrix for a residue contact map.

Args:

- `index` (`numpy.ndarray`): Array containing contact indices.
- `protid` (`str`): Protein identifier.

Returns: `tuple` — (`matrix`, `stats`, `chainstats`). For single chains, `chainstats` is an empty `dict`.

```
get_stats(mat)
```

Module: `functions/calculating/get_stats.py`

Calculates the percentage of entangled contacts (Parallel and Cross) further along the diagonal.

Args:

- `mat` (`numpy.ndarray`): The topological relationship matrix.

Returns: `numpy.ndarray` — An array containing the percentage of entangled contacts for each diagonal.

```
length_filter(index, distance, mode='<')
```

Module: functions/calculating/length_filter.py

Filters contact indices based on sequence separation distance.

Args:

- `index` (`numpy.ndarray`): Array of contact indices.
- `distance` (`int`): The distance threshold for filtering.
- `mode` (`str`, optional): The filtering mode ('`<`' for less than or equal to, '`>`' for greater than or equal to). Defaults to '`<`'.

Returns: `numpy.ndarray` — The filtered array of contact indices.

```
local_ct(index, mat, numbering)
```

Module: functions/calculating/local_ct.py

Calculates local circuit topology statistics for each residue.

Args:

- `index` (`numpy.ndarray`): Array of contact indices.
- `mat` (`numpy.ndarray`): The topological relationship matrix.
- `numbering` (`list` or `numpy.ndarray`): List of residue numbers/identifiers.

Returns: `dict` — Keys are residue indices; values are dicts containing counts of '`P`' (Parallel), '`IP`' (Inverse Parallel), '`X`' (Cross), and '`S`' (Series) contacts.

Plotting Functions

```
circuit_plot(index, protid, numbering)
```

Module: functions/plots/circuit_plot.py

Plots the circuit topology of a protein as a series of arcs.

Args:

- `index` (`numpy.ndarray`): Array of contact indices.
- `protid` (`str`): Protein identifier.
- `numbering` (`list` or `numpy.ndarray`): List of residue numbers/identifiers.

Returns: `None`

```
local_topology_plot(index, mat, numbering, siteid, relation)
```

Module: functions/plots/local_topology_plot.py

Plots three arc diagrams highlighting contacts for a specific residue site: all contacts, contacts matching the specified relation type (coloured blue), and the selected site contacts (coloured red) against a grey background.

Args:

- `index` (`numpy.ndarray`): Array of contact indices.
- `mat` (`numpy.ndarray`): The topological relationship matrix.
- `numbering` (`list` or `numpy.ndarray`): List of residue numbers/identifiers.
- `siteid` (`int`): The residue index to highlight.
- `relation` (`str`): Contact relation type to highlight ('`X`' , '`S`' , '`P`' , or '`IP`').

Returns: `None`

```
matrix_plot(mat, protid)
```

Module: functions/plots/matrix_plot.py

Plots the topological relationship matrix for a single chain.

Args:

- `mat` (`numpy.ndarray`): The topological relationship matrix.
- `protid` (`str`): Protein identifier.

Returns: None

`matrix_plot_model(mat, protid)`

Module: functions/plots/matrix_plot_model.py

Plots the topological relationship matrix for a whole model (multiple chains).

Args:

- `mat` (`numpy.ndarray`): The topological relationship matrix.
- `protid` (`str`): Protein identifier.

Returns: None

`autopct_func(pct)`

Module: functions/plots/stats_plot.py

Formats a percentage value for display in a pie chart. Returns a formatted string if the value is $\geq 1.0\%$, otherwise returns an empty string.

Args:

- `pct` (`float`): The percentage value.

Returns: `str` — Formatted percentage string (e.g. "12.3%") or "" .

`stats_plot(entangled, psc, protid)`

Module: functions/plots/stats_plot.py

Plots the fraction of entangled contacts versus distance from the diagonal and a pie chart of contact types.

Args:

- `entangled` (`numpy.ndarray`): Array of entangled contact fractions per diagonal offset.
- `psc` (`Sequence[float]`): List of contact type counts (P, S, C, and optionally I2/I3/I4/T2/T3/L for multi-chain).
- `protid` (`str`): Protein identifier used as the figure title.

Returns: None

Importing Functions

`retrieve_chain(input_file, chainid=0)`

Module: functions/importing/retrieve_chain.py

Retrieves a specific chain from a PDB or MMCIF file. Removes hetero-residues, DNA/RNA, and unknown residues before returning.

Args:

- `input_file` (`Path`): Path to the input PDB or MMCIF file.
- `chainid` (`int` or `str`, optional): The chain ID to retrieve. Can be an integer index or a string chain ID. Defaults to 0 .

Returns: `tuple` — (chain, protid) where `chain` is a `Bio.PDB.Chain.Chain` object and `protid` is a `str` .

Exporting Functions

`export_cmap3(index, protid, numbering, output_dir)`

Module: functions/exporting/export_cmap3.py

Exports a residue contact map (as a binary matrix) to a CSV file.

Args:

- `index` (`numpy.ndarray`): Array of contact indices.
- `protid` (`str`): Protein identifier (used as the filename stem).
- `numbering` (`Sequence[int]`): List of residue numbers/identifiers.
- `output_dir` (`Path`): The directory to save the CSV file. Created if it does not exist.

Returns: None

`export_mat(index, mat, protid, output_dir)`

Module: functions/exporting/export_mat.py

Exports the topological relationship matrix to a CSV file. Handles both single-chain (2-column index) and multi-chain (4-column index) formats.

Args:

- `index` (`numpy.ndarray`): Array of contact indices.
- `mat` (`numpy.ndarray`): The topological relationship matrix.
- `protid` (`str`): Protein identifier (used as the filename stem).
- `output_dir` (`Path`): The directory to save the CSV file. Created if it does not exist.

Returns: None

`export_psc(psclist, output_dir)`

Module: functions/exporting/export_psc.py

Exports the counts of Parallel, Series, and Cross contacts (and others) to a CSV file named `pscreresults.csv`.

Args:

- `psclist` (`Sequence[Sequence[object]]`): A list of PSC statistics rows. Each row begins with a `protid` string followed by numeric counts.
- `output_dir` (`Path`): The directory to save the CSV file. Created if it does not exist.

Returns: None

Analysis Functions

`run_local_ct(self)`

Module: analysis/local_ct_analysis.py

Runs the local circuit topology analysis based on user-selected parameters. Handles data retrieval, calculation, plotting, and exporting.

Args:

- `self`: The main GUI class instance (`CTDialog`).

Returns: None

`run_multi_analysis(self)`

Module: analysis/multiple_file_analysis.py

Runs the multi-file circuit topology analysis. Iterates through PDB/CIF files in the selected input directory, performs analysis on each, and generates plots and/or exports as configured.

Args:

- `self`: The main GUI class instance (`CTDialog`).

Returns: None

`run_standard_analysis(self)`

Module: analysis/single_file_analysis.py

Runs the standard single-file circuit topology analysis. Handles data retrieval, calculation, plotting, and exporting. Supports both single-chain and multi-chain objects.

Args:

- `self`: The main GUI class instance (`CTDialog`).

Returns: None

```
run_single_frame_analysis(self)
```

Module: analysis/single_frame_analysis.py

Runs circuit topology analysis for a single frame of a trajectory or a single PDB file from a directory.

Args:

- `self`: The main GUI class instance (`CTDialog`).

Returns: None

```
toggle_frame_controls(self, enabled)
```

Module: analysis/single_frame_analysis.py

Toggles the enabled state of the frame selector spinbox and the "Analyze selected trajectory frame" button.

Args:

- `self`: The main GUI class instance (`CTDialog`).
- `enabled` (`bool`): `True` to enable the controls, `False` to disable them.

Returns: None

```
visualize_molecule(self, contact_type)
```

Module: analysis/visualization.py

Visualizes the circuit topology on the selected molecule in PyMOL by coloring residues based on contact density for the chosen contact type.

Args:

- `self`: The main GUI class instance (`CTDialog`).
- `contact_type` (`str`): The type of contact to visualize (`'P'` , `'S'` , or `'X'`).

Returns: None

Utility Functions

```
clear_selected_local_file(self)
```

Module: utils/clear_file.py

Clears the currently selected local file, removes the associated PyMOL object, and resets the relevant UI controls.

Args:

- `self`: The main GUI class instance.

Returns: None

```
clear_selected_single_file(self)
```

Module: utils/clear_file.py

Clears the currently selected single file, removes the associated PyMOL object, and resets the relevant UI controls.

Args:

- `self`: The main GUI class instance.

Returns: None

```
_choose_output_dir(self, attr_name, label)
```

Module: utils/directory.py

Shared helper: opens a directory dialog and stores the chosen path as an attribute on `self`.

Args:

- `self`: The main GUI class instance.
- `attr_name ( str )`: Attribute name on `self` where the chosen path is stored.
- `label ( QLabel )`: Label widget to update with the elided path.

Returns: None

`_load_structure_file(self, label, attr_file, attr_obj)`

Module: `utils/directory.py`

Shared helper: opens a file dialog for a PDB or CIF file, loads it into PyMOL, and checks for non-polymer atoms.

Args:

- `self`: The main GUI class instance.
- `label ( QLabel )`: Label widget to update with the elided file path.
- `attr_file ( str )`: Attribute name on `self` for storing the file path.
- `attr_obj ( str )`: Attribute name on `self` for storing the PyMOL object name.

Returns: None

`choose_file(self)`

Module: `utils/directory.py`

Opens a file dialog to select a single input file for the single-file analysis tab and loads it into PyMOL.

Args:

- `self`: The main GUI class instance.

Returns: None

`choose_input_dir_multi(self)`

Module: `utils/directory.py`

Opens a directory dialog to select the input directory containing PDB files for multi-file analysis. Updates the frame selector spinbox based on the number of PDB files found.

Args:

- `self`: The main GUI class instance.

Returns: None

`choose_local_file(self)`

Module: `utils/directory.py`

Opens a file dialog to select a local PDB or CIF file for the local circuit topology analysis tab and loads it into PyMOL.

Args:

- `self`: The main GUI class instance.

Returns: None

`choose_local_output_dir(self)`

Module: `utils/directory.py`

Opens a directory dialog to select the output directory for the local analysis tab.

Args:

- `self`: The main GUI class instance.

Returns: None

`choose_output_dir(self)`

Module: `utils/directory.py`

Opens a directory dialog to select the output directory for the single-file analysis tab.

Args:

- `self` : The main GUI class instance.

Returns: `None`

`choose_output_dir_multi(self)`

Module: `utils/directory.py`

Opens a directory dialog to select the output directory for the multi-file analysis tab.

Args:

- `self` : The main GUI class instance.

Returns: `None`

`set_label_text_elided(file_path, label)`

Module: `utils/directory.py`

Sets the text of a `QLabel` to an elided (middle-truncated) version of the file path when it exceeds the available label width.

Args:

- `file_path (str)`: The full file or directory path.
- `label (QLabel)`: The label widget to update.

Returns: `None`

`get_folding_score(mat, index, numbering)`

Module: `utils/folding_score.py`

Calculates the CT folding score as the sum of mean per-residue densities for parallel, series, and crossover contact types.

Args:

- `mat (numpy.ndarray)`: The topological relationship matrix.
- `index (numpy.ndarray)`: Array of contact indices.
- `numbering (numpy.ndarray)`: Array of residue numbers/identifiers.

Returns: `float` — The calculated folding score.

`get_local_values(self)`

Module: `utils/get_values.py`

Retrieves parameters for local analysis from the GUI.

Args:

- `self` : The main GUI class instance.

Returns: `dict` — Contains keys: `output_directory`, `cutoff_distance`, `cutoff_numcontacts`, `exclude_neighbour`, `local_topology_plot`, `res_id`, `contact_type`, `local_ct`, `export_cmap3`, `export_mat`.

`get_multiple_values(self)`

Module: `utils/get_values.py`

Retrieves parameters for multi-file analysis from the GUI.

Args:

- `self` : The main GUI class instance.

Returns: `dict` — Contains keys: `directory`, `traj_directory`, `output_directory`, `cutoff_distance`, `cutoff_numcontacts`, `exclude_neighbour`, `circuit_plot`, `matrix_plot`, `stats_plot`, `export_cmap3`, `export_mat`, `export_psc`, `plot_psc`, `length_filtering`, `filtering_distance`,

`length_filter_mode`, `energy_filtering`, `energy_filtering_mode`.

`get_values(self)`

Module: `utils/get_values.py`

Retrieves parameters for single-file analysis from the GUI.

Args:

- `self`: The main GUI class instance.

Returns: `dict` — Contains keys: `output_directory`, `cutoff_distance`, `cutoff_numcontacts`, `exclude_neighbour`, `circuit_plot`, `folding_score`, `matrix_plot`, `export_cmap3`, `export_mat`.

`get_vis_vals(self)`

Module: `utils/get_values.py`

Retrieves visualization parameters from the GUI.

Args:

- `self`: The main GUI class instance.

Returns: `dict` — Contains keys: `cutoff_distance`, `cutoff_numcontacts`, `exclude_neighbour`.

`_poll_pymol_objects(self)`

Module: `utils/helpers.py`

Single poll that refreshes both object dropdowns (single-file and local) from one `cmd.get_object_list()` call.

Args:

- `self`: The main GUI class instance.

Returns: `None`

`init_timers(self)`

Module: `utils/helpers.py`

Initializes a `QTimer` that fires every 2 seconds to poll PyMOL for updated object lists.

Args:

- `self`: The main GUI class instance.

Returns: `None`

`make_info_button(tooltip)`

Module: `utils/helpers.py`

Creates a small flat info button (ⓘ) with a tooltip.

Args:

- `tooltip` (`str`): The text to display in the tooltip.

Returns: `QPushButton` — The configured info button.

`make_param_row(label_text, tooltip, spinbox)`

Module: `utils/helpers.py`

Creates a standard horizontal parameter row containing a label, an info button, a stretch, and a spinbox.

Args:

- `label_text ( str )`: Text for the row label.
- `tooltip ( str )`: Tooltip text for the info button.
- `spinbox ( QAbstractSpinBox )`: The spinbox widget.

Returns: `QHBoxLayout` — The assembled row layout.

`object_exists(name)`

Module: `utils/helpers.py`

Checks if a PyMOL object with the given name exists in the current session.

Args:

- `name ( str )`: The name of the object.

Returns: `bool` — `True` if the object exists, `False` otherwise.

`temp_pdb_export(selection, state=None)`

Module: `utils/helpers.py`

Context manager that saves a PyMOL selection to a temporary PDB file, yields the `Path`, then deletes the file on exit.

Args:

- `selection ( str )`: PyMOL selection string.
- `state ( int , optional)`: PyMOL state to save. Defaults to the current state.

Yields: `Path` — Path to the temporary PDB file.

`update_chain_combo_box(self)`

Module: `utils/helpers.py`

Clears and repopulates the chain combo box with the chains available in the currently selected object.

Args:

- `self`: The main GUI class instance.

Returns: `None`

`has_non_polymer_atoms()`

Module: `utils/non_polymer.py`

Checks if there are any non-polymer atoms in the current PyMOL session.

Returns: `bool` — `True` if non-polymer atoms exist, `False` otherwise.

`new_file_has_non_polymer_atoms(obj_name)`

Module: `utils/non_polymer.py`

Checks if a specific PyMOL object contains non-polymer atoms.

Args:

- `obj_name ( str )`: The name of the object to check.

Returns: `bool` — `True` if the object contains non-polymer atoms, `False` otherwise.

`remove_non_polymer_atoms()`

Module: `utils/non_polymer.py`

Removes all non-polymer atoms from the PyMOL session and refreshes the view.

Returns: `None`

`show_warning_dialog(self)`

Module: `utils/non_polymer.py`

Shows a warning dialog before removing non-polymer atoms. Offers Continue, Cancel, and "Don't show again" options. Suppresses future warnings for the session if the user selects "Don't show again".

Args:

- `self`: The main GUI class instance.

Returns: `None`

`handle_local_object_change(self, obj_name)`

Module: `utils/object_change.py`

Handles changes to the selected object in the local analysis tab. Checks for non-polymer atoms, warns the user if found, and updates the residue range.

Args:

- `self`: The main GUI class instance.
- `obj_name` (`str`): The name of the newly selected object.

Returns: `None`

`handle_standard_object_change(self, obj_name)`

Module: `utils/object_change.py`

Handles changes to the selected object in the standard analysis tab. Checks for non-polymer atoms and warns the user if found.

Args:

- `self`: The main GUI class instance.
- `obj_name` (`str`): The name of the newly selected object.

Returns: `None`

`get_residue_range(self, obj_name)`

Module: `utils/residues.py`

Retrieves the residue range for each chain in the specified object and updates the chain combo box and residue ID spinbox.

Args:

- `self`: The main GUI class instance.
- `obj_name` (`str`): The name of the object to analyze.

Returns: `None`

`update_residue_range(self)`

Module: `utils/residues.py`

Updates the residue ID spinbox range and value based on the currently selected chain.

Args:

- `self`: The main GUI class instance.

Returns: `None`

`color_by_topology(molecule_name, topology_vector, numbering, topology_type)`

Module: `utils/topology.py`

Colors a PyMOL object based on a topology vector using a contact-type-specific color spectrum.

Args:

- `molecule_name` (`str`): The name of the PyMOL object to color.
- `topology_vector` (`numpy.ndarray`): Per-residue topology density values.
- `numbering` (`list` or `numpy.ndarray`): Residue numbers corresponding to the vector.
- `topology_type` (`str`): Contact type being visualized (`'P'` , `'S'` , or `'X'`), used to select the color palette.

Returns: `None`

`get_topology_vector(mat, index, topology_type, numbering)`

Module: `utils/topology.py`

Calculates a topology vector representing the per-residue density of a specific contact type.

Args:

- `mat` (`numpy.ndarray`): The topological relationship matrix.
- `index` (`numpy.ndarray`): Array of contact indices.
- `topology_type` (`str`): The type of topology to calculate (`'P'` , `'S'` , or `'X'`).
- `numbering` (`list` or `numpy.ndarray`): List of residue numbers/identifiers.

Returns: `numpy.ndarray` or `None` — The calculated topology vector, or `None` if an invalid `topology_type` is supplied.

`export_frames_from_traj(self)`

Module: `utils/trajectory.py`

Exports each frame of the loaded trajectory as a separate PDB file. Prompts the user for confirmation and an output directory before writing.

Args:

- `self` : The main GUI class instance.

Returns: `None`

`select_mol_file(self)`

Module: `utils/trajectory.py`

Opens a file dialog to select a structure file (PDB or CIF) for trajectory analysis and loads it into PyMOL.

Args:

- `self` : The main GUI class instance.

Returns: `None`

`select_xtc_file(self)`

Module: `utils/trajectory.py`

Opens a file dialog to select a trajectory file (XTC, DCD, TRR, NC), loads it into PyMOL via `cmd.load_traj` , and removes non-polymer atoms.

Args:

- `self` : The main GUI class instance.

Returns: `None`

`update_list(self, new_objects=None)`

Module: `utils/updates.py`

Updates the list of available objects in the single-file analysis dropdown.

Args:

- `self` : The main GUI class instance.
- `new_objects` (`list[str]` , optional): Pre-fetched object list. If `None` , queries PyMOL directly.

Returns: `None`

`update_local_list(self, new_objects=None)`

Module: `utils/updates.py`

Updates the list of available objects in the local analysis dropdown.

Args:

- `self` : The main GUI class instance.
- `new_objects` (`list[str]` , optional): Pre-fetched object list. If `None` , queries PyMOL directly.

Returns: `None`

`update_output_widgets(self)`

Module: `utils/updates.py`

Updates visibility of output directory widgets for single-file export options based on checkbox state.

Args:

- `self` : The main GUI class instance.

Returns: `None`

`update_output_widgets_local(self)`

Module: `utils/updates.py`

Updates the visibility of output directory widgets in the local analysis tab based on checkbox states.

Args:

- `self` : The main GUI class instance.

Returns: `None`

`update_output_widgets_multi(self)`

Module: `utils/updates.py`

Updates the visibility of output directory widgets in the multi-file analysis tab based on checkbox states.

Args:

- `self` : The main GUI class instance.

Returns: `None`

GUI Functions

`__init_plugin__(app=None)`

Module: `__init__.py`

Initialize the plugin within PyMOL. Invoked by PyMOL on plugin load. Attempts direct registration, then falls back to platform-specific dependency installation (conda/pip). Adds the plugin to the PyMOL plugins menu on success.

Args:

- `app` (optional): The PyMOL application instance (unused, accepted for API compatibility).

Returns: `None`

`run_plugin_gui()`

Module: `__init__.py`

Create or raise the plugin GUI dialog. Returns the singleton `CTDialog` instance; creates it if it does not yet exist or has been closed.

Returns: `CTDialog`

`__init__(self, parent=None)`

Module: `gui_class.py`

Type: CTDialo**g** method

Constructs the dialog and performs UI initialization. Sets window title, geometry, and stylesheet, then calls `init_ui()` and `init_timers()` .

`_poll_pymol_objects(self)`

Module: `gui_class.py`

Type: CTDialo**g** method

Single timer callback that refreshes both object dropdowns by delegating to `_poll_pymol_objects` in `utils/helpers.py` .

`choose_file(self)`

Module: `gui_class.py`

Type: CTDialo**g** method

Open a file chooser to select a single input file (single-file tab).

`choose_input_dir_multi(self)`

Module: `gui_class.py`

Type: CTDialo**g** method

Open a directory chooser for selecting multiple input files (multi-file).

`choose_local_file(self)`

Module: `gui_class.py`

Type: CTDialo**g** method

Open a file chooser to select a local PDB/file for local analysis.

`choose_local_output_dir(self)`

Module: `gui_class.py`

Type: CTDialo**g** method

Open a directory chooser for specifying the output folder (local analysis).

`choose_output_dir(self)`

Module: `gui_class.py`

Type: CTDialo**g** method

Open a directory chooser for specifying the output folder (single-file).

`choose_output_dir_multi(self)`

Module: `gui_class.py`

Type: CTDialo**g** method

Open a directory chooser for specifying the output folder (multi-file).

`clear_selected_local_file(self)`

Module: gui_class.py

Type: CTDialog method

Clear the selected local file control value(s).

`clear_selected_single_file(self)`

Module: gui_class.py

Type: CTDialog method

Clear the selected single-file control value(s).

`export_frames_from_traj(self)`

Module: gui_class.py

Type: CTDialog method

Export frames from an opened trajectory according to UI parameters.

`get_local_values(self)`

Module: gui_class.py

Type: CTDialog method

Return settings from the local analysis tab UI controls.

`get_multiple_values(self)`

Module: gui_class.py

Type: CTDialog method

Return settings from the multi-file analysis tab UI controls.

`get_residue_range(self, obj_name=None)`

Module: gui_class.py

Type: CTDialog method

Request the residue range for an object and update the UI. If `obj_name` is `None`, reads the current value from the local object dropdown.

`get_values(self)`

Module: gui_class.py

Type: CTDialog method

Return settings from the single-file tab UI controls.

`get_vis_vals(self)`

Module: gui_class.py

Type: CTDialog method

Return visualization-specific values from the UI.

`handle_local_object_change(self, obj_name=None)`

Module: gui_class.py

Type: CDialog method

Handle changes in the local object dropdown. If `obj_name` is `None`, reads the current value from the dropdown.

`handle_standard_object_change(self, obj_name=None)`

Module: `gui_class.py`

Type: CDialog method

Handle changes in the standard object dropdown. If `obj_name` is `None`, reads the current value from the dropdown.

`init_local_tab(self)`

Module: `gui_class.py`

Type: CDialog method

Initialize widgets and layout for the local analysis tab.

`init_multi_file_tab(self)`

Module: `gui_class.py`

Type: CDialog method

Initialize widgets and layout for the multi-file analysis tab.

`init_single_file_tab(self)`

Module: `gui_class.py`

Type: CDialog method

Initialize widgets and layout for the single-file analysis tab.

`init_timers(self)`

Module: `gui_class.py`

Type: CDialog method

Initialize any repeating timers used by the GUI (e.g., polling PyMOL state).

`init_ui(self)`

Module: `gui_class.py`

Type: CDialog method

Create the top-level tab widget and initialize each feature tab.

`run_local_ct(self)`

Module: `gui_class.py`

Type: CDialog method

Run local circuit-topology analysis using current local-tab settings.

`run_multi_analysis(self)`

Module: `gui_class.py`

Type: CDialog method

Run the batch multi-file analysis flow.

`run_single_frame_analysis(self)`

Module: `gui_class.py`

Type: `CTDialog` method

Run an analysis for the currently selected single frame.

`run_standard_analysis(self)`

Module: `gui_class.py`

Type: `CTDialog` method

Run the standard single-file analysis flow using current UI settings.

`select_mol_file(self)`

Module: `gui_class.py`

Type: `CTDialog` method

Select a molecular file used by the trajectory tools.

`select_xtc_file(self)`

Module: `gui_class.py`

Type: `CTDialog` method

Select an XTC (trajectory) file used by the trajectory tools.

`show_warning_dialog(self)`

Module: `gui_class.py`

Type: `CTDialog` method

Display a warning dialog when a non-polymeric selection is detected.

`toggle_frame_controls(self, enabled)`

Module: `gui_class.py`

Type: `CTDialog` method

Enable or disable UI controls that affect frame selection.

Args:

- `enabled (bool)`: Whether the frame-related controls should be enabled.
-

`update_chain_combo_box(self)`

Module: `gui_class.py`

Type: `CTDialog` method

Refresh the chain selection combobox based on the currently selected object.

`update_list(self)`

Module: `gui_class.py`

Type: CTDialog method

Refresh the object list shown in the single-file tab.

`update_local_list(self)`

Module: gui_class.py

Type: CTDialog method

Refresh the object list shown in the local-analysis tab.

`update_output_widgets(self)`

Module: gui_class.py

Type: CTDialog method

Update widgets that display or depend on the output path (single-file).

`update_output_widgets_local(self)`

Module: gui_class.py

Type: CTDialog method

Update widgets that display or depend on the output path (local-analysis).

`update_output_widgets_multi(self)`

Module: gui_class.py

Type: CTDialog method

Update widgets that display or depend on the output path (multi-file).

`update_residue_range(self)`

Module: gui_class.py

Type: CTDialog method

Apply the current residue-range widget values to the model or internal state.

`visualize_molecule(self, contact_type)`

Module: gui_class.py

Type: CTDialog method

Trigger molecule visualization in PyMOL.

Args:

- `contact_type` (str): Identifier describing the visualization type to render ('P' , 'S' , or 'X').
-

`init_local_tab(self)`

Module: tabs/local_tab.py

Initializes the 'Local Circuit Topology' tab in the GUI. Sets up widgets for file selection, parameter input, analysis options, and exporting.

Args:

- `self` : The main GUI class instance.

Returns: None

```
init_multi_file_tab(self)
```

Module: tabs/multiple_file_tab.py

Initializes the 'Multi-File Analysis' tab in the GUI. Sets up widgets for directory selection, trajectory handling, parameter input, analysis options, and exporting.

Args:

- `self` : The main GUI class instance.

Returns: None

```
init_single_file_tab(self)
```

Module: tabs/single_file_tab.py

Initializes the 'Single-File Analysis' tab in the GUI. Sets up widgets for file selection, parameter input, analysis options, visualization, and exporting.

Args:

- `self` : The main GUI class instance.

Returns: None

Initialization Functions

```
check_installed_packages(requirements_list)
```

Module: initialization_checks.py

Checks if the required packages are installed in the current environment.

Args:

- `requirements_list` (`list[str]`): List of package names to check.

Returns: `tuple[bool, list[str]]` — `True` and the list of installed packages if all are present; `False` and the list of missing packages otherwise.

```
conda_init(user_install)
```

Module: initialization_checks.py

Initializes conda for PowerShell, using admin or user privileges depending on `user_install`.

Args:

- `user_install` (`bool`): `True` if PyMOL is installed in a user directory (no admin required), `False` otherwise.

Returns: `bool` — `True` if initialization was successful, `False` otherwise.

```
get_requirements(req_path)
```

Module: initialization_checks.py

Parses the `requirements.yml` file to get a list of required package names.

Args:

- `req_path` (`Path`): Path to the requirements YAML file.

Returns: `list[str]` — A list of package names.

Raises: `FileNotFoundError` if the file does not exist; `ValueError` on parse errors.

```
install_failed(reqs=REQUIREMENTS_FILE)
```

Module: initialization_checks.py

Logs instructions for manual installation of dependencies if all automated installation strategies fail.

Args:

- `reqs` (`Path`, optional): Path to the requirements file. Defaults to `REQUIREMENTS_FILE`.

Returns: `None`

`is_conda_installed()`

Module: `initialization_checks.py`

Checks if conda is installed on the system by running `where conda` (Windows) or `which conda` (Unix).

Returns: `tuple[bool, Path | None]` — A boolean indicating whether conda was found and the path to the conda executable (`None` if not found).

`is_path_user(path)`

Module: `initialization_checks.py`

Checks if a given path is within the user's home directory.

Args:

- `path (Path | None)`: The path to check.

Returns: `bool` — `True` if the path is within the user's home directory, `False` otherwise.

`linux_install(reqs=REQUIREMENTS_FILE)`

Module: `initialization_checks.py`

Performs dependency installation on Linux using `pip`.

Args:

- `reqs (Path, optional)`: Path to the requirements file. Defaults to `REQUIREMENTS_FILE`.

Returns: `bool` — `True` if all packages were installed successfully, `False` otherwise.

`mac_install(reqs=REQUIREMENTS_FILE)`

Module: `initialization_checks.py`

Performs dependency installation on macOS using `pip`.

Args:

- `reqs (Path, optional)`: Path to the requirements file. Defaults to `REQUIREMENTS_FILE`.

Returns: `bool` — `True` if installation was successful, `False` otherwise.

`pymol_install(reqs=REQUIREMENTS_FILE)`

Module: `initialization_checks.py`

Attempts to install dependencies using the PyMOL terminal and conda by issuing `conda init` and `conda env update` commands via `cmd.do`.

Args:

- `reqs (Path, optional)`: Path to the requirements file. Defaults to `REQUIREMENTS_FILE`.

Returns: `bool` — `True` if the install commands were issued without error, `False` if the requirements file was not found.

`register_pymol_functions()`

Module: `initialization_checks.py`

Registers the plugin's core functions as PyMOL commands by importing each function and calling `cmd.extend`. Covers plotting, calculating, importing, and exporting functions.

Returns: `None`

`win_install(reqs=REQUIREMENTS_FILE)`

Module: `initialization_checks.py`

Performs dependency installation on Windows using PowerShell and conda. Requests admin privileges via `Start-Process -Verb RunAs` when conda is installed system-wide.

Args:

- `reqs` (`Path`, optional): Path to the requirements file. Defaults to `REQUIREMENTS_FILE`.

Returns: `bool` — `True` if installation was successful, `False` otherwise.

Last Updated: May 12, 2026